2.11 uniqueness

This helper validates that the attribute's value is unique right before the object gets saved. It does not create a uniqueness constraint in the database, so it may happen that two different database connections create two records with the same value for a column that you intend to be unique. To avoid that, you must create a unique index on that column in your database.

```
class Account < ActiveRecord::Base
  validates :email, uniqueness: true
end
```

The validation happens by performing an SQL query into the model's table, searching for an existing record with the same value in that attribute.

There is a `:scope` option that you can use to specify other attributes that are used to limit the uniqueness check:

```
class Holiday < ActiveRecord::Base
  validates :name, uniqueness: { scope: :year,
    message: "should happen once per year" }
end
```

Should you wish to create a database constraint to prevent possible violations of a uniqueness validation using the `:scope` option, you must create a unique index on both columns in your database. See the MySQL manual for more details about multiple column indexes or the PostgreSQL manual for examples of unique constraints that refer to a group of columns.

There is also a `:case_sensitive` option that you can use to define whether the uniqueness constraint will be case sensitive or not. This option defaults to true.

```
class Person < ActiveRecord::Base
  validates :name, uniqueness: { case_sensitive: false }
end
```

Note that some databases are configured to perform case-insensitive searches anyway.

The default error message is *"has already been taken"*.

2.12 validates_with

This helper passes the record to a separate class for validation.

```
class GoodnessValidator < ActiveModel::Validator
  def validate(record)
    if record.first_name == "Evil"
      record.errors[:base] << "This person is evil"
    end
  end
end

class Person < ActiveRecord::Base
  validates_with GoodnessValidator
end
```

Errors added to `record.errors[:base]` relate to the state of the record as a whole, and not to a specific attribute.

The `validates_with` helper takes a class, or a list of classes to use for validation. There is no default error message for `validates_with`. You must manually add errors to the record's errors collection in the validator class.

To implement the validate method, you must have a `record` parameter defined, which is the record to be validated.